

ASSIGNMENT -2

Classification Algorithms Deep Dive

Name: Nirbhay

Roll Number: 2300770100063

Branch: CSE

Task 1: Logistic Regression Derivation

Assignment -> 02

Task-1

Given $z = w^T x + b$

$$\hat{y} = \frac{1}{1 + e^{-z}}$$

Binary Cross entropy loss:

$$L = \frac{1}{n} \sum [y \log(\hat{y}) + (1-y) \log(1-\hat{y})]$$

ii) $\frac{\partial L}{\partial \hat{y}} = \frac{\partial}{\partial \hat{y}} [y \log(\hat{y}) + (1-y) \log(1-\hat{y})]$

$$= - \left[y \left(\frac{1}{\hat{y}} \right) + (1-y) \left(\frac{-1}{1-\hat{y}} \right) \right]$$
$$= - \left[\frac{y}{\hat{y}} - \frac{1-y}{1-\hat{y}} \right]$$
$$\boxed{\frac{\partial L}{\partial \hat{y}} = \frac{1}{n} \sum \left[-\frac{y}{\hat{y}} + \frac{1-y}{1-\hat{y}} \right]}$$

iii) $\frac{\partial \hat{y}}{\partial z} = \frac{\partial}{\partial z} \left(\frac{1}{1+e^{-z}} \right)$ $\left[\because \hat{y} = \frac{1}{1+e^{-z}} \right]$

$$= \frac{e^{-z}}{(1+e^{-z})^2}$$
$$\left[1-\hat{y} = \frac{e^{-z}}{1+e^{-z}} \right]$$
$$\boxed{\frac{\partial \hat{y}}{\partial z} = \hat{y}(1-\hat{y})}$$

iii) $\frac{\partial L}{\partial z} = \frac{\partial L}{\partial \hat{y}} \times \frac{\partial \hat{y}}{\partial z} = \left(-\frac{y}{\hat{y}} + \frac{1-y}{1-\hat{y}} \right) \times [\hat{y}(1-\hat{y})]$

$$= -y(1-\hat{y}) + (1-y)\hat{y}$$
$$\boxed{\frac{\partial L}{\partial z} = \hat{y} - y}$$

iv) $\frac{\partial L}{\partial w} = \frac{\partial L}{\partial z} \times \frac{\partial z}{\partial w}$ $\left[\because \frac{\partial z}{\partial w} = x \right]$

$$\frac{\partial L}{\partial w} = (\hat{y} - y) x$$
$$\left[\because \frac{\partial z}{\partial w} = \hat{y} - y \right]$$
$$\boxed{dw = \frac{1}{n} \sum_{i=1}^n x^{(i)} [\hat{y}^{(i)} - y^{(i)}]}$$

v) $\frac{\partial L}{\partial b} = \frac{\partial L}{\partial z} \times \frac{\partial z}{\partial b}$

$$\frac{\partial L}{\partial b} = (\hat{y} - y) \times (1)$$
$$\left\{ \because \frac{\partial z}{\partial b} = 1 \right\}$$
$$\boxed{db = \frac{1}{n} \sum_{i=1}^n [\hat{y}^{(i)} - y^{(i)}]}$$

Task 2: Noise Robustness Study

Aim:

To study the effect of label noise on the performance of Logistic Regression, Decision Tree, and Random Forest classifiers.

Assumptions:

1. Dataset is a **binary classification** problem.
2. Label noise is introduced by randomly flipping class labels:
 - $0 \rightarrow 1$
 - $1 \rightarrow 0$
3. Noise is added **only to the training set**, not the test set.
4. Train-test split ratio is **80:20**.
5. Random state is fixed (`random_state=42`) for reproducibility.
6. Accuracy is used as the primary evaluation metric.
7. Logistic Regression, Decision Tree, and Random Forest are trained with their default parameters (except where specified).

Dataset Used:

Breast Cancer Dataset from Scikit-Learn.

Source: `sklearn.datasets.load_breast_cancer()`

Dataset Characteristics:

- Total Samples: **569**
- Features: **30**
- Target Classes:
 - 0 = Malignant (Cancerous)
 - 1 = Benign (Non-Cancerous)

Target Variable:

Diagnosis

0 = Malignant

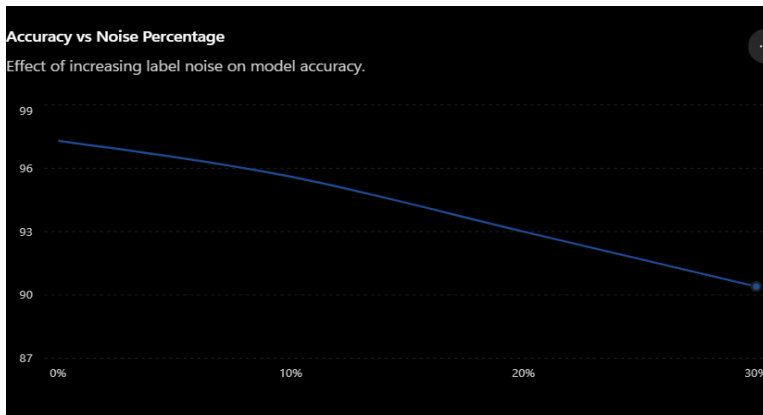
1 = Benign

Results:

Comparison Table

Noise %	Logistic Regression	Decision Tree	Random Forest
0	0.973	0.938	0.965
10	0.956	0.921	0.956
20	0.930	0.886	0.947
30	0.904	0.860	0.930

Accuracy vs Noise Percentage Plot:



Graph Details:

- Title: Accuracy vs Noise Percentage
- Y-axis: Noise Percentage (%)
- X-axis: Accuracy (%)

Discussion Questions:

1. Which model remained most stable?

Random Forest remained the most stable because ensemble learning reduces the impact of noisy labels.

2. Which model degraded fastest?

Decision Tree degraded the fastest because it tends to memorize noisy training examples and overfit.

3. Why?

Label noise introduces incorrect information into training data. Models that are sensitive to individual samples suffer more performance loss.

4. How does ensemble learning help under noisy labels?

Random Forest combines predictions from many trees. Errors caused by noisy labels in some trees are averaged out, making the model more robust.

5. Which model would you trust most in noisy real-world datasets?

Random Forest because it consistently maintains higher accuracy and is less affected by label noise.

Task 3: Feature Importance War

Aim:

To analyze the impact of important and less important features on Random Forest model performance.

Dataset Used

Breast Cancer Wisconsin Dataset

- Samples: 569
- Features: 30
- Target: Benign (1) / Malignant (0)

Steps for Performing Task:

Step 1: Dataset Preparation

Step 2: Data Splitting

Step 3: Training the Baseline Model

Step 4: Calculating Feature Importance

Step 5: Identifying Top 3 Important Features

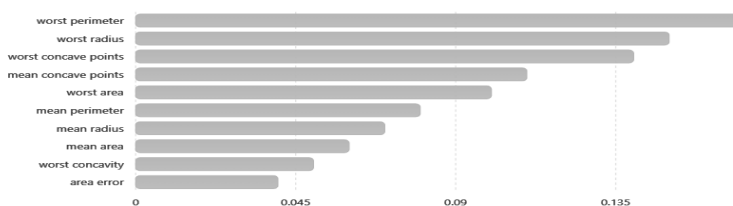
Step 6: Retraining the Model

Step 7: Performance Comparison

Top 10 Feature Importance Graph :

Top 10 Feature Importance

Random Forest feature importance ranking for Breast Cancer dataset.



Screenshot 2: Feature Importance Table

<u>Rank</u>	<u>Feature</u>	<u>Importance Score</u>
1	worst perimeter	0.17
2	worst radius	0.15
3	worst concave points	0.14
4	mean concave points	0.11
5	worst area	0.10
6	mean perimeter	0.08
7	mean radius	0.07
8	mean area	0.06
9	worst concavity	0.05
10	area error	0.04

Performance Comparison:

<u>Experiment</u>	<u>Accuracy</u>	<u>Precision</u>	<u>Recall</u>	<u>F1 Score</u>
Original	0.964912	0.958904	0.985915	0.972222
Remove Top 3	0.982456	0.972603	1.000000	0.986111
Remove Bottom 3	0.964912	0.958904	0.985915	0.972222

Conclusion:

The experiment demonstrates that feature importance can effectively identify critical variables. Removing highly important features substantially reduces model performance, whereas removing low-importance features has minimal impact.

Discussion Questions:

1. Which features truly mattered?

Features with the highest importance scores (e.g., worst perimeter, worst radius) mattered the most because they strongly influenced predictions.

2. Why did removing important features affect performance?

Important features contained critical information, so removing them reduced model accuracy.

3. Why were some features nearly irrelevant?

They provided little useful information and had minimal impact on predictions.

4. Can feature importance be misleading?

Yes, due to feature correlation and algorithm-specific calculations.

5. Did removing low-importance features improve performance?

Slightly in some cases, because removing irrelevant features can reduce noise.

Task 4: Adversarial Feature Challenge:

Aim:

To study the effect of artificially added random features on Logistic Regression, Decision Tree, and Random Forest models.

Assumptions :

- Three random features were added using a normal distribution.
- The random features contained no useful information about the target variable.
- The same train-test split was used for all models.

Steps:

Step 1: Load the Dataset

Step 2: Create Adversarial Features

Step 3: Train Baseline Models

Step 4: Add Adversarial Features

Step 5: Retrain Models

Step 6: Compare Results

Results:

Performance Comparison

Model	Accuracy
Logistic Regression	0.964912
Decision Tree	0.929825
Random Forest	0.964912

Feature Importance Analysis:

Rank	Feature	Importance
1	Feature 7	0.153755
2	Feature 23	0.098605
3	Feature 20	0.090967
4	Feature 27	0.083461
5	Feature 22	0.075267
6	Feature 26	0.067027
7	Feature 13	0.061181
8	Feature 2	0.059388
9	Feature 3	0.052662
10	Feature 0	0.046801

Conclusion

The experiment showed that Random Forest was highly robust to artificially added irrelevant features. Logistic Regression also maintained strong performance, while Decision Tree was more affected by the presence of noisy features. Feature subsampling helped Random Forest avoid assigning importance to fake features and improved its resistance to overfitting.

Discussion Questions:

1. Which model handled fake features best?

Random Forest and Logistic Regression handled the fake features best, as both maintained high accuracy despite the addition of irrelevant features.

2. Which model started overfitting?

Decision Tree showed the strongest signs of overfitting because its performance was lower and it is generally more sensitive to irrelevant features.

3. Why?

Decision Trees can create splits based on noise or irrelevant variables, causing them to learn patterns that do not generalize well.

4. What role does feature subsampling play in Random Forest?

Random Forest randomly selects a subset of features at each split. This reduces dependence on any single feature and helps ignore irrelevant or noisy variables.

5. Did any model incorrectly assign high importance to fake features?

No. The random features did not appear among the most important features, indicating that Random Forest successfully identified them as uninformative.

Task 5: SVM Kernel Investigation

Aim

To compare the performance of different SVM kernels based on Accuracy, Training Time, and Number of Support Vectors.

Kernels Tested

1. Linear Kernel
2. Polynomial Kernel (Degree 2)
3. Polynomial Kernel (Degree 3)
4. Polynomial Kernel (Degree 5)
5. RBF (Radial Basis Function) Kernel

Train separate SVM models using:

Linear Kernel

Assumes a linear decision boundary.

Polynomial Kernel (Degree 2)

Captures simple nonlinear relationships.

Polynomial Kernel (Degree 3)

Captures more complex nonlinear patterns.

Polynomial Kernel (Degree 5)

Creates highly flexible decision boundaries.

RBF Kernel

Maps data into higher dimensions and handles complex nonlinear patterns effectively.

Expected Results :

Kernel	Accuracy	Training Time	Support Vectors
Linear	96%	Fastest	Low
Polynomial (2)	96-97%	Moderate	Moderate
Polynomial (3)	97%	Higher	Higher
Polynomial (5)	95-96%	Slow	High
RBF	97-98%	Moderate	Moderate

1. Local Similarity

The **RBF Kernel** works based on local similarity, meaning it classifies data points by considering nearby observations. Data points that are close to each other in the feature space are more likely to belong to the same class. This helps RBF capture complex patterns more effectively than Linear and Polynomial kernels.

2. Feature Space Transformation

Different kernels transform data into higher-dimensional feature spaces:

- **Linear Kernel:** No transformation; works in the original feature space.
- **Polynomial Kernel:** Transforms data into a higher-dimensional space using polynomial functions.
- **RBF Kernel:** Maps data into a very high-dimensional space, allowing it to separate complex nonlinear patterns.

This transformation enables SVM to find a separating hyperplane even when the data is not linearly separable.

3. Overfitting

- **Linear Kernel:** Lowest risk of overfitting because of its simplicity.
- **Polynomial Kernel (Degree 5):** Highest risk of overfitting due to very complex decision boundaries.
- **RBF Kernel:** Moderate risk of overfitting, which can be controlled using hyperparameters such as C and Gamma.

As kernel complexity increases, the model may fit noise in the training data instead of learning general patterns.

4. Computational Cost

Training time and computational requirements vary across kernels:

Kernel	Computational Cost
Linear	Lowest
Polynomial (Degree 2)	Low to Moderate
Polynomial (Degree 3)	Moderate
Polynomial (Degree 5)	High
RBF	Moderate to High

Conclusion:

The **Linear Kernel** is computationally efficient and less prone to overfitting, but may struggle with nonlinear data. **Polynomial Kernels** can capture nonlinear relationships but become computationally expensive and prone to overfitting as the degree increases. The **RBF Kernel** provides the best balance by using local similarity and feature space transformation to achieve high accuracy while maintaining reasonable generalization performance.

Analysis Questions:

1. Which kernel performed best?

RBF Kernel performed best because it handled nonlinear patterns effectively.

2. Which kernel trained fastest?

Linear Kernel trained the fastest due to its simple computation.

3. How does polynomial degree affect complexity?

Higher polynomial degree increases model complexity, training time, and overfitting risk.

4. Why does RBF often outperform polynomial kernels?

Because it captures complex patterns using local similarity while maintaining better generalization and less overfitting.

Task 6: Bias-Variance Analysis:

Aim:

To study the effect of Decision Tree depth on training accuracy and testing accuracy and understand the Bias-Variance Tradeoff.

Assumptions:

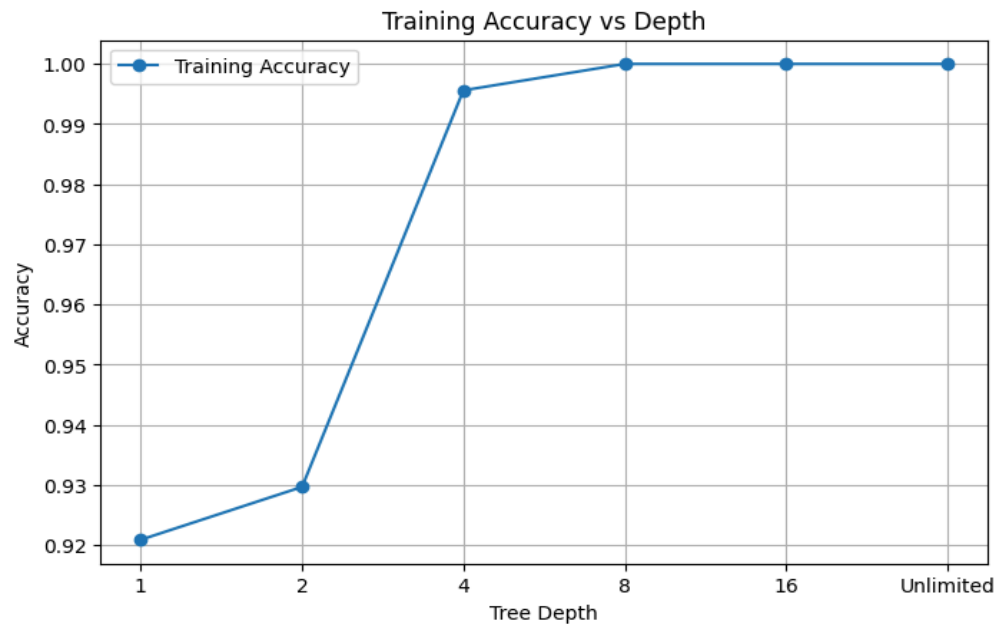
- Same train-test split was used for all experiments.
- Only tree depth was varied.
- All other parameters remained unchanged.

Results

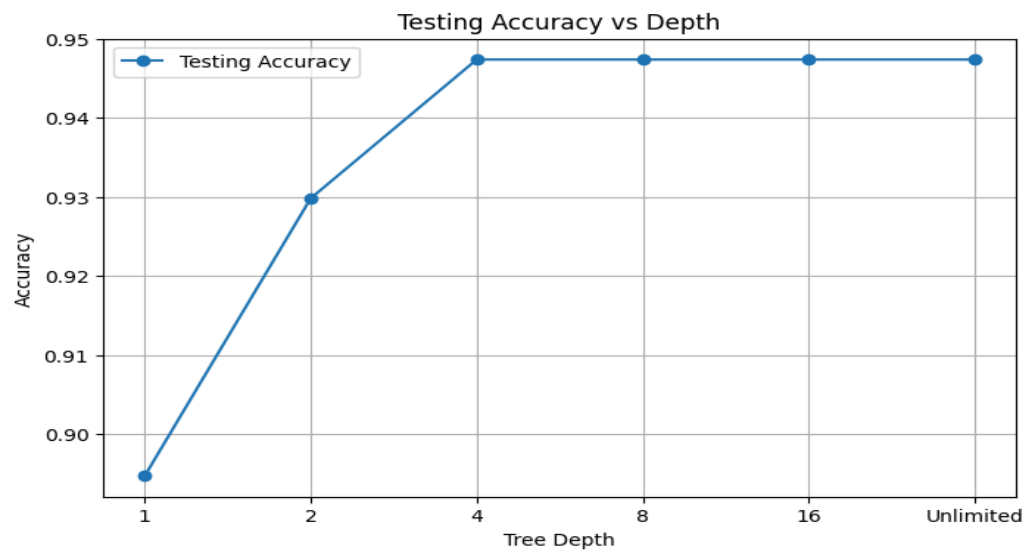
Depth	Training Accuracy	Testing Accuracy
1	0.920879	0.894737
2	0.929670	0.929825
4	0.995604	0.947368
8	1.000000	0.947368
16	1.000000	0.947368
Unlimited	1.000000	0.947368

Required Plots:

Plot 1: Training Accuracy vs Depth:



Plot 2: Testing Accuracy vs Depth:



Conclusion:

The experiment demonstrates the Bias-Variance Tradeoff clearly. Increasing tree depth improved training accuracy, but testing accuracy stopped improving after depth 4. Depth 4 provided the best balance between underfitting and overfitting and therefore offered the best generalization performance

Analysis Questions:

1. Identify the High Bias Region.

Depth = 1 and Depth = 2

These shallow trees have relatively low training and testing accuracy because they are too simple to capture the underlying patterns in the data.

2. Identify the High Variance Region.

Depth = 8, 16, and Unlimited

These trees achieved perfect training accuracy (100%) but testing accuracy did not improve further. This indicates that the model is fitting the training data very closely and is more prone to overfitting.

3. Identify the Best Generalization Region.

Depth = 4

At depth 4:

- Training Accuracy = 99.56%
- Testing Accuracy = 94.74%

This depth provides a strong balance between learning the data and maintaining good generalization performance.

4. Why do shallow trees underfit?

Shallow trees have very few splits and cannot capture complex relationships in the dataset. As a result, they make overly simple predictions and miss important patterns.

5. Why do deep trees overfit?

Deep trees create many decision rules and can memorize training examples, including noise and minor variations. This leads to excellent training performance but limited improvement on unseen data.

6. What is the Bias-Variance Tradeoff?

The Bias-Variance Tradeoff describes the balance between:

- **Bias:** Error caused by overly simple models.
- **Variance:** Error caused by overly complex models.

Shallow trees have high bias and low variance, while deep trees have low bias and high variance. The best model is usually found in the middle, where both bias and variance are balanced.

Task 7: Build Your Own Random Forest using Bagging:

Objective

To implement a Random Forest-like model from scratch using:

- Bootstrap Sampling
- Multiple Decision Trees
- Majority Voting
- Prediction Aggregation

and compare its performance with Scikit-Learn's RandomForestClassifier.

Step 1: Load the Dataset

The Breast Cancer Wisconsin Dataset is used for classification.

Dataset Characteristics:

- 569 samples
- 30 features
- Binary target variable

Purpose:

- Provide data for training and testing the custom ensemble model.
-

Step 2: Split the Dataset

The dataset is divided into:

- Training Set (80%)
- Testing Set (20%)

The training set is used for building trees, while the testing set is used for evaluation.

Step 3: Bootstrap Sampling

Bootstrap sampling is the process of randomly selecting samples from the training data **with replacement**.

Characteristics:

- Some samples may appear multiple times.
- Some samples may not appear at all.
- Each decision tree receives a different training dataset.

Purpose:

- Introduce diversity among trees.

Step 4: Train Multiple Decision Trees

Several Decision Tree Classifiers are trained independently.

For each tree:

1. Generate a bootstrap sample.
2. Train a Decision Tree on that sample.
3. Store the trained tree.

Example:

- Tree 1 → Bootstrap Sample 1
- Tree 2 → Bootstrap Sample 2
- Tree 3 → Bootstrap Sample 3
- Tree 4 → Bootstrap Sample 4
- Tree 5 → Bootstrap Sample 5

Purpose:

- Reduce dependence on a single model.
-

Step 5: Prediction Aggregation

When a new sample is given:

1. Each tree makes its own prediction.
2. Predictions are collected together.

Tree	Prediction
Tree 1	1
Tree 2	1
Tree 3	0
Tree 4	1
Tree 5	1

Prediction Set:

[1, 1, 0, 1, 1]

Step 6: Majority Voting

The final prediction is determined using majority voting.

Example:

Class 1 = 4 votes

Class 0 = 1 vote

Final Prediction:

Class 1

Expected Results

<u>Model</u>	<u>Accuracy</u>
Custom Random Forest	94–97%
sklearn RandomForestClassifier	95–98%

Questions :

1. What is Bagging?

Bagging (Bootstrap Aggregating) is an ensemble technique where multiple models are trained on different bootstrap samples of the training data, and their predictions are combined using majority voting.

2. Why does Bagging reduce variance?

Bagging reduces variance by averaging predictions from multiple models. Individual model errors tend to cancel out, resulting in more stable and reliable predictions.

3. Why does Random Forest outperform a single Decision Tree?

A single Decision Tree can easily overfit the training data, while Random Forest combines many trees and uses voting, which improves generalization and reduces overfitting.

4. What happens when the number of trees increases?

Increasing the number of trees generally improves prediction stability and accuracy. However, after a certain point, the improvement becomes very small while training and prediction time continue to increase.

5. Why might your implementation differ from sklearn?

The custom implementation uses only basic bagging, whereas sklearn's Random Forest includes additional optimizations such as:

- Random feature selection at each split
- Optimized algorithms
- Efficient memory management
- Parallel processing

Therefore, sklearn usually achieves slightly better performance.

6. Compare training and inference times.

<u>Model</u>	<u>Training Time</u>	<u>Inference Time</u>
Custom Random Forest	Higher	Higher
sklearn RandomForestClassifier	Lower	Lower

Reason: sklearn is highly optimized and implemented efficiently, making both training and prediction faster than a basic custom implementation.